Data Structures and Algorithms

Lecture 15

Heap and Heap Sort

Dept. of ICT, Faculty of Technology- UoR, 2026

Objectives

- Study on heap data Structure
- Study on Heap sorting techniques

Heap Data Structure

Introduction

- The heap is a binary tree, meaning at the most, each parent has two children.
- The heap is an abstract data type.
- The heap data structure is a very useful data structure that every programmer should know well.
- The heap data structure is used behind the scenes to perform the **heap sort**.
- Understanding how the heap works will allow programmers to make wiser decisions when programming in an environment where memory management is crucial.

Applications of HEAP Data Structures

- To quickly find the smallest and largest element from a collection of items or array.
- In the implementation of Priority queue in graph algorithms
- In order to overcome the Worst Case Complexity of Quick Sort algorithm from O(n^2) to O(nlog(n)) in Heap Sort.
- For finding the order in statistics.
- Systems concerned with security and embedded system such as Linux Kernel uses Heap Sort because of the O(nlog(n)) .

Properties of Heap data structure

A heap is a binary tree with two special properties :

- It must have all its nodes in a specific order.

  Nodes must be arranged in an order according to values based on **Max heap** or **Min heap**.

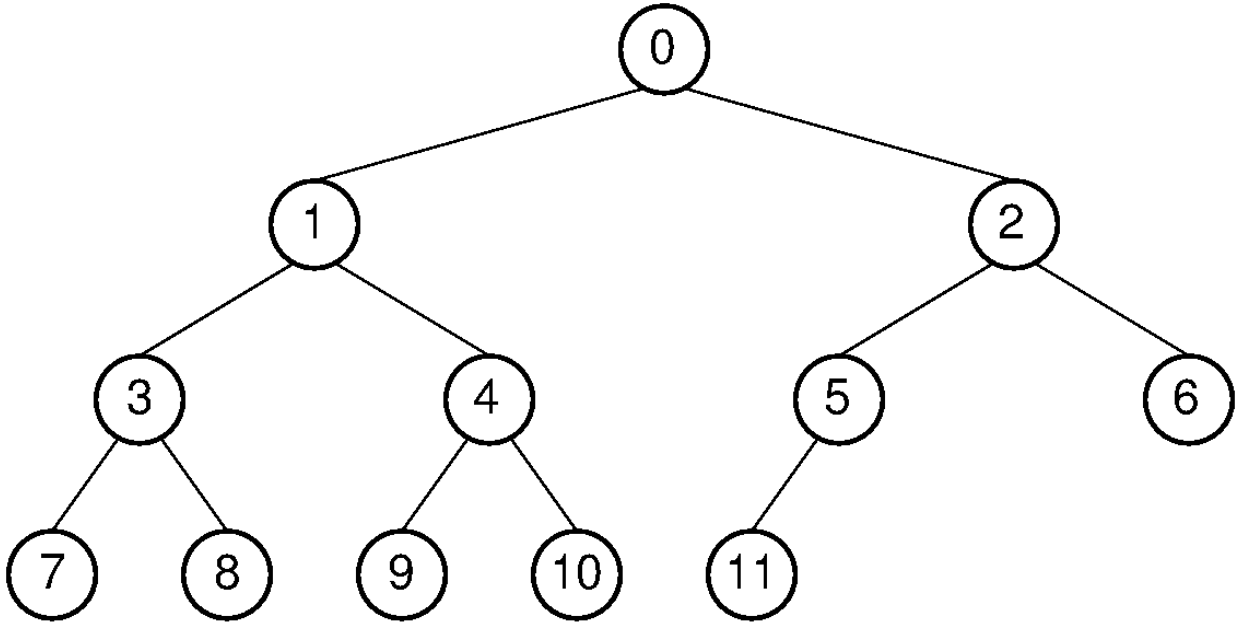
- Its **shape** must be completed.

  Node are arranged based on their value.

  All levels in a heap must full, except last level and nodes must be filled from left to right strictly.

What is a complete binary tree?

A Binary Tree is complete Binary Tree if all levels are filled except possibly the last level and the last level has all keys as left as possible



(a)

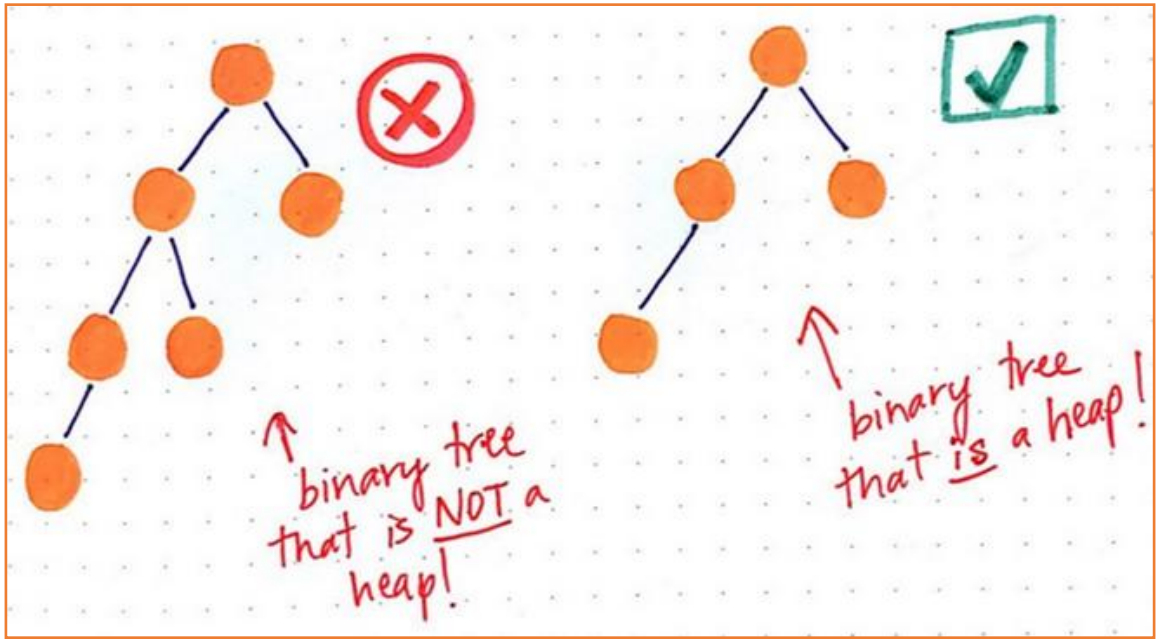
Two rules of heaping

Rule 1: Shape

A heap must be a complete binary tree.

This means all the levels of the tree must be completely filled.

Except may be the last one.



Two rules of heaping

Rule 2: Order

- A heap's root must have all of its children be either greater than or equal to its children (Min-heap)

  or

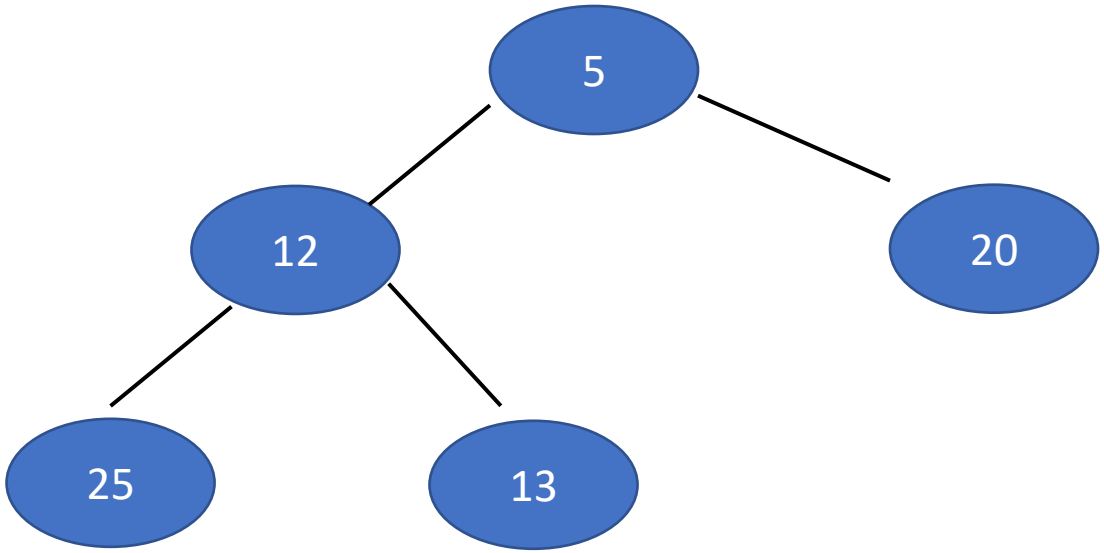
- Less than or equal to its children (Max-Heap).

In both above cases, last node can be alone.

Min heap

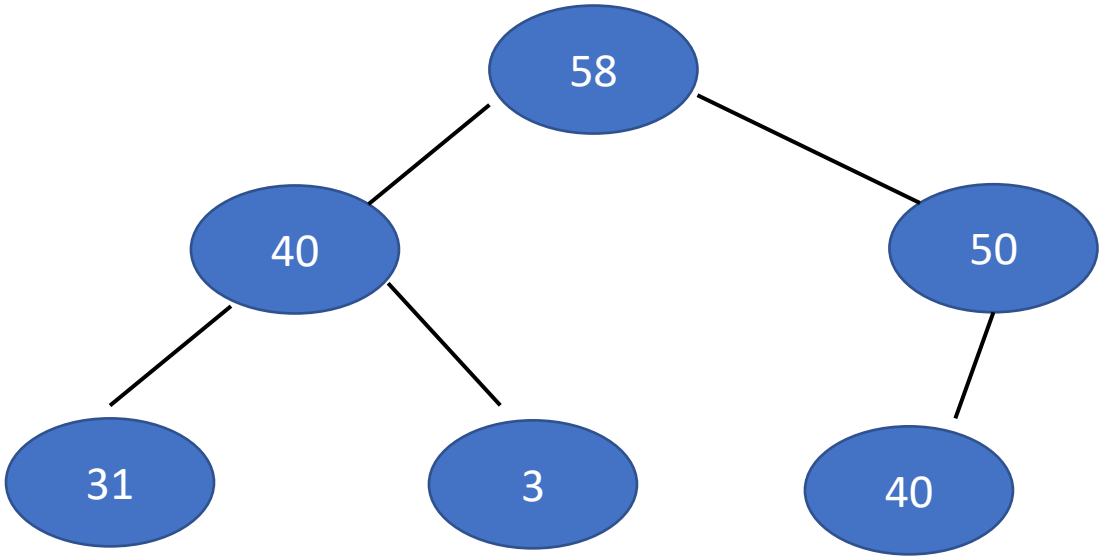
- Every single parent node (Including the root node) is less than or equal to the value of its children node.

- The minimum value key is always at root node.

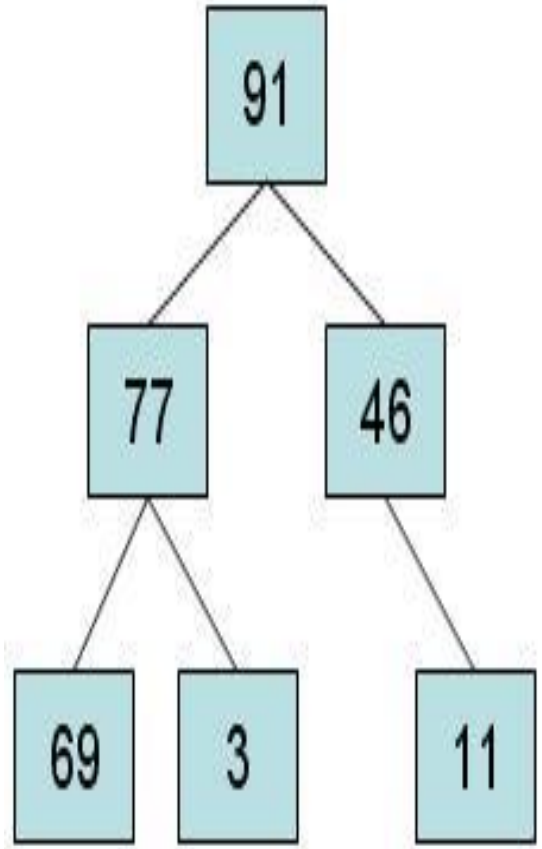


Max heap

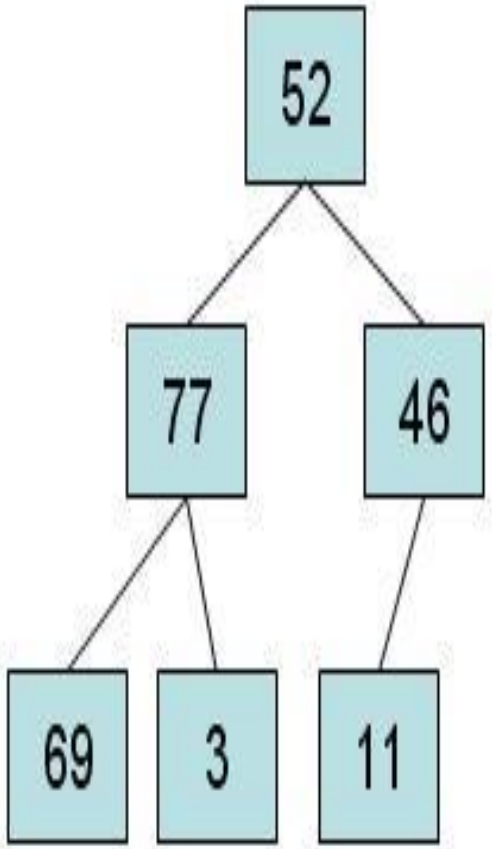
- Every single parent node (including the root) is greater than or equal to its children
- The maximum value key is always at root node.



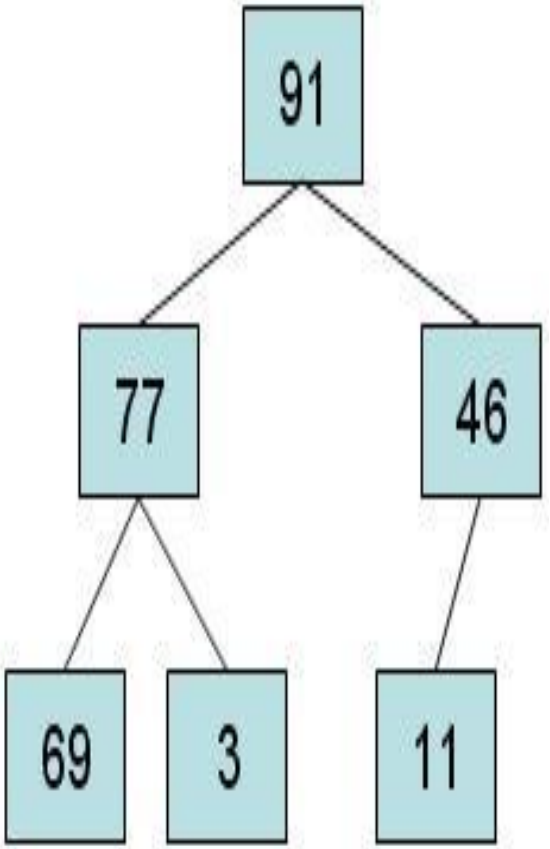
Which one is a heap?



A

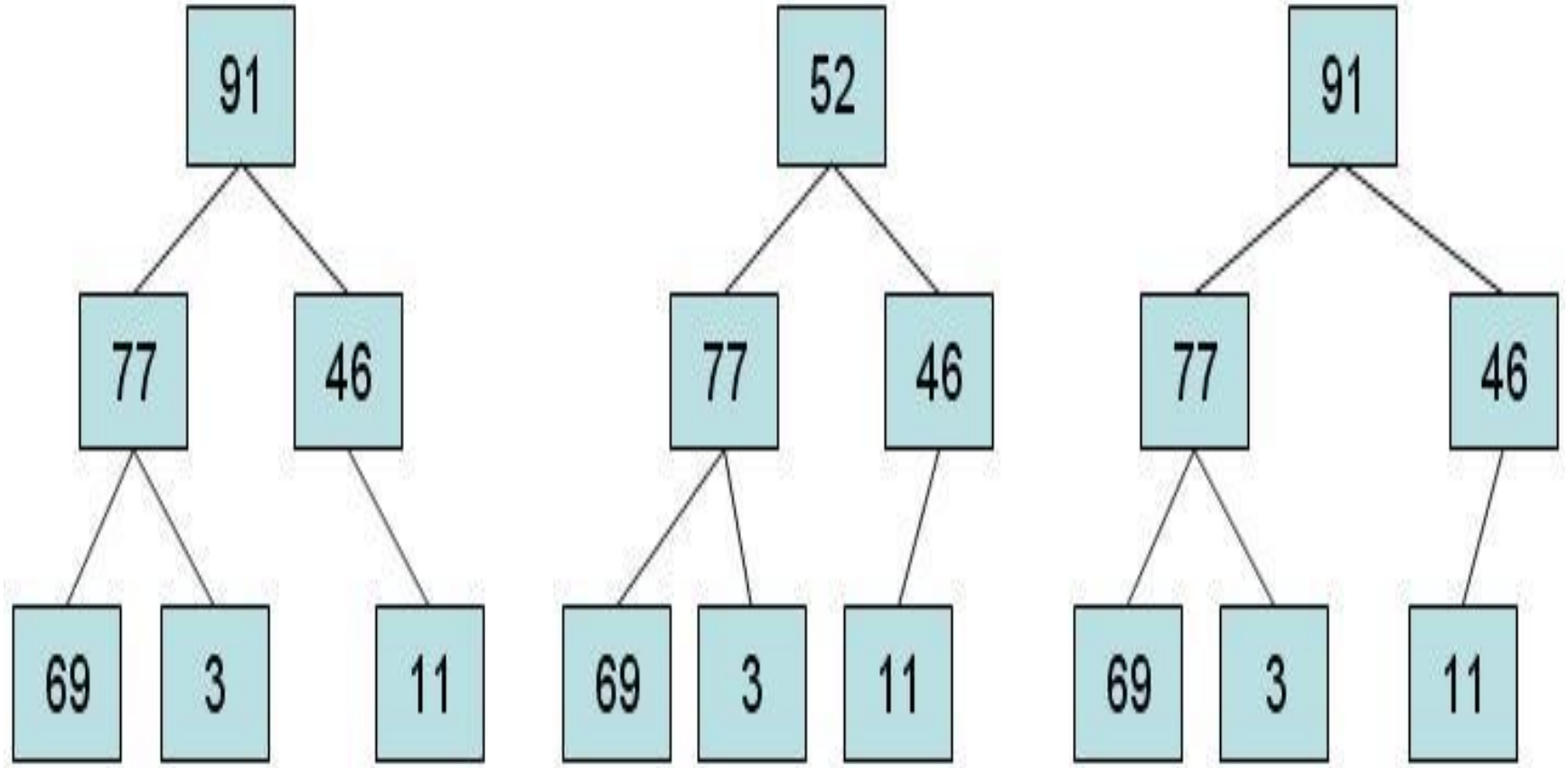


B



C

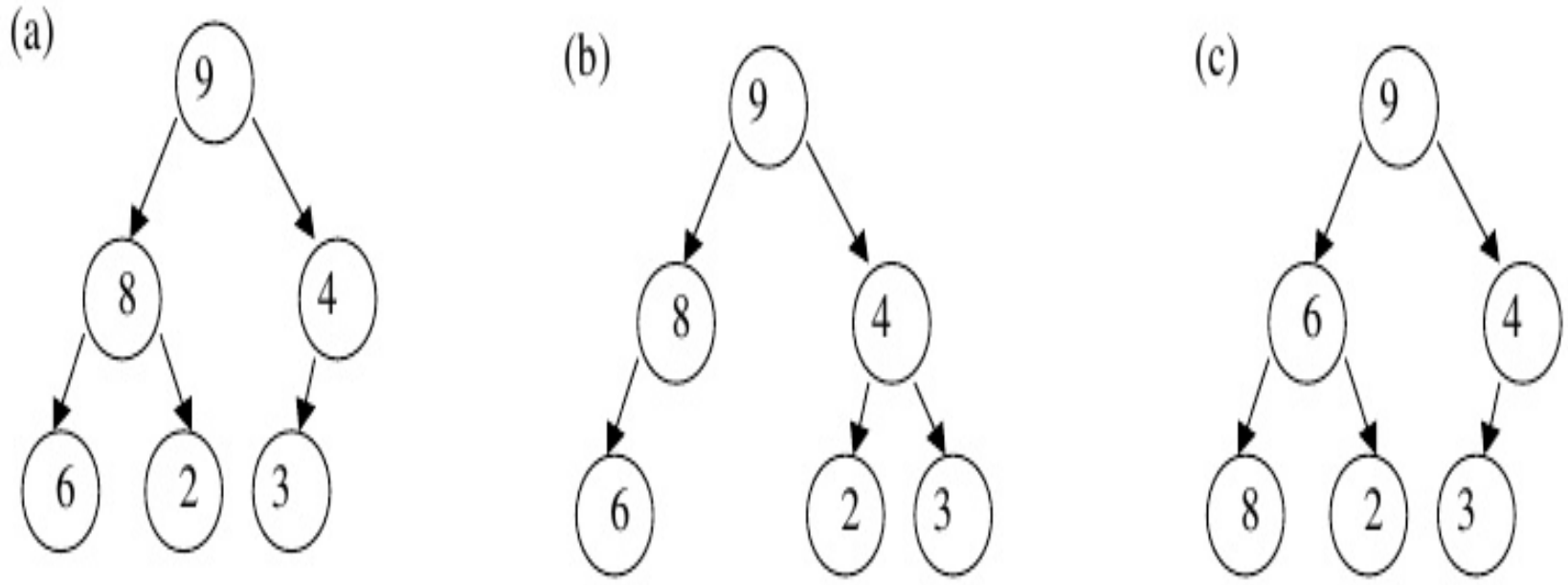
Which one is a heap?



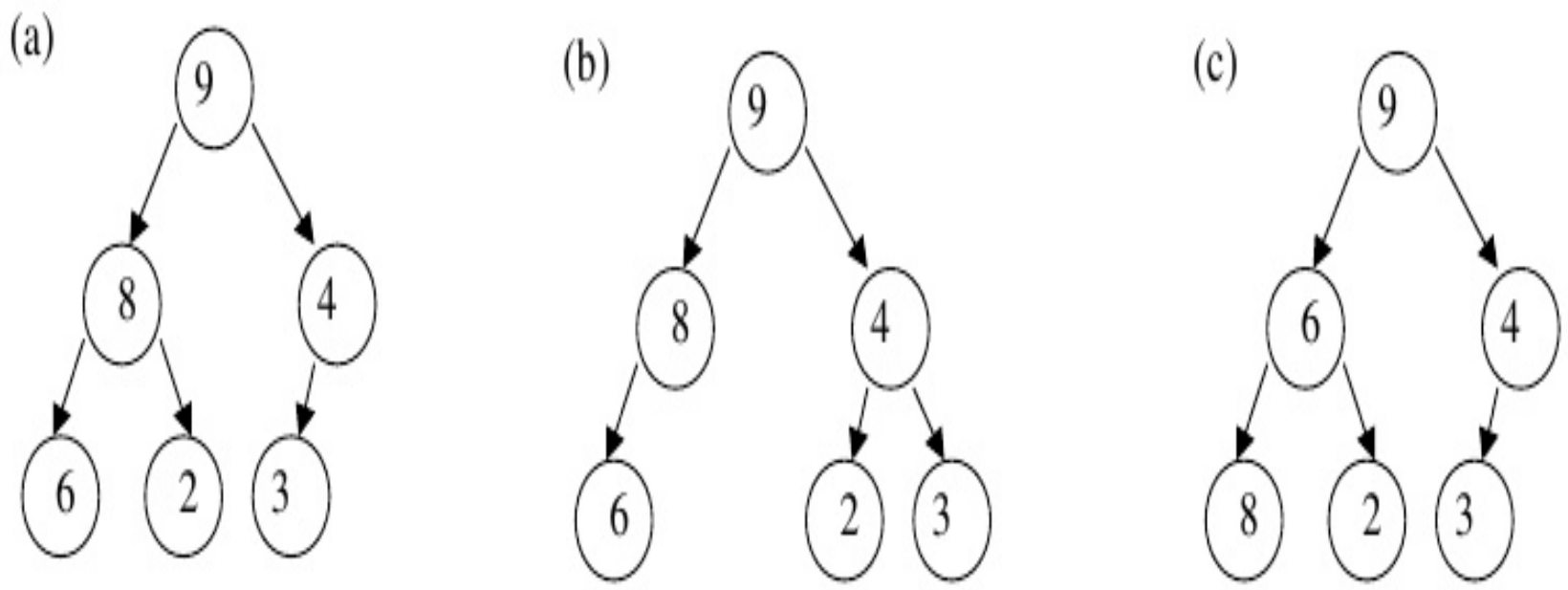
A B C

Part C is heap binary tree

Which one is a heap?



Which one is a heap?



In the above example only (a) is a heap.(b) is not a heap as it is not complete and (c) is complete but does not satisfy the second property defied for heaps

Important things to note in Heap

1. In heap, we can have Duplicate values - No restriction against that.
2. Heap doesn’t follow the rules of a Binary Search tree.(Binary search tree: left node does not have to be smaller than the right node.)
3. Ordering of the child nodes is not important for a heap(Only ordering that matter is the heap-order property, or the ordering of parent nodes compared to the children.)

Operations of heaps

1. **Insertion (Growing)**
2. **Deletion (Shrinking)**
3. **Finding Max and Min**

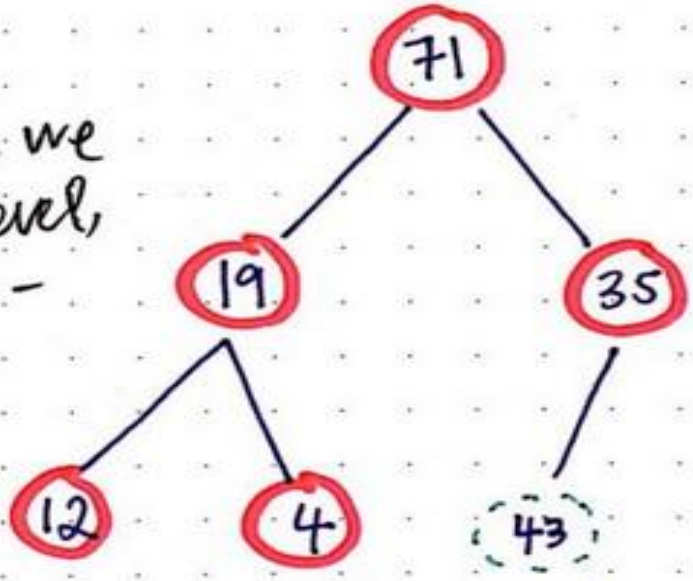
The crucial thing to always keep in mind when growing or shrinking a heap is how we are manipulating or changing its structure in the process.

We must always maintain the shape and structure of a heap — otherwise, we're violating one of its two properties!

Insertion operation (Growing the heap)

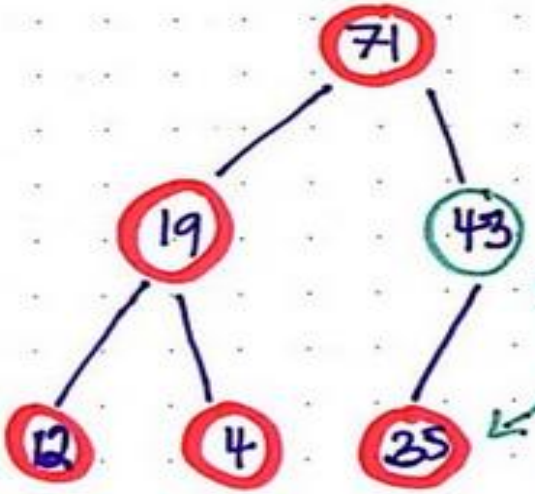
When growing a heap, we can only ever add a node to the left-most available spot in the tree; that is to say, t**he left most available node**, at the lowest possible level

In order to add → 43 this element to the heap, we must find the lowest level, and add it to the left-most available spot



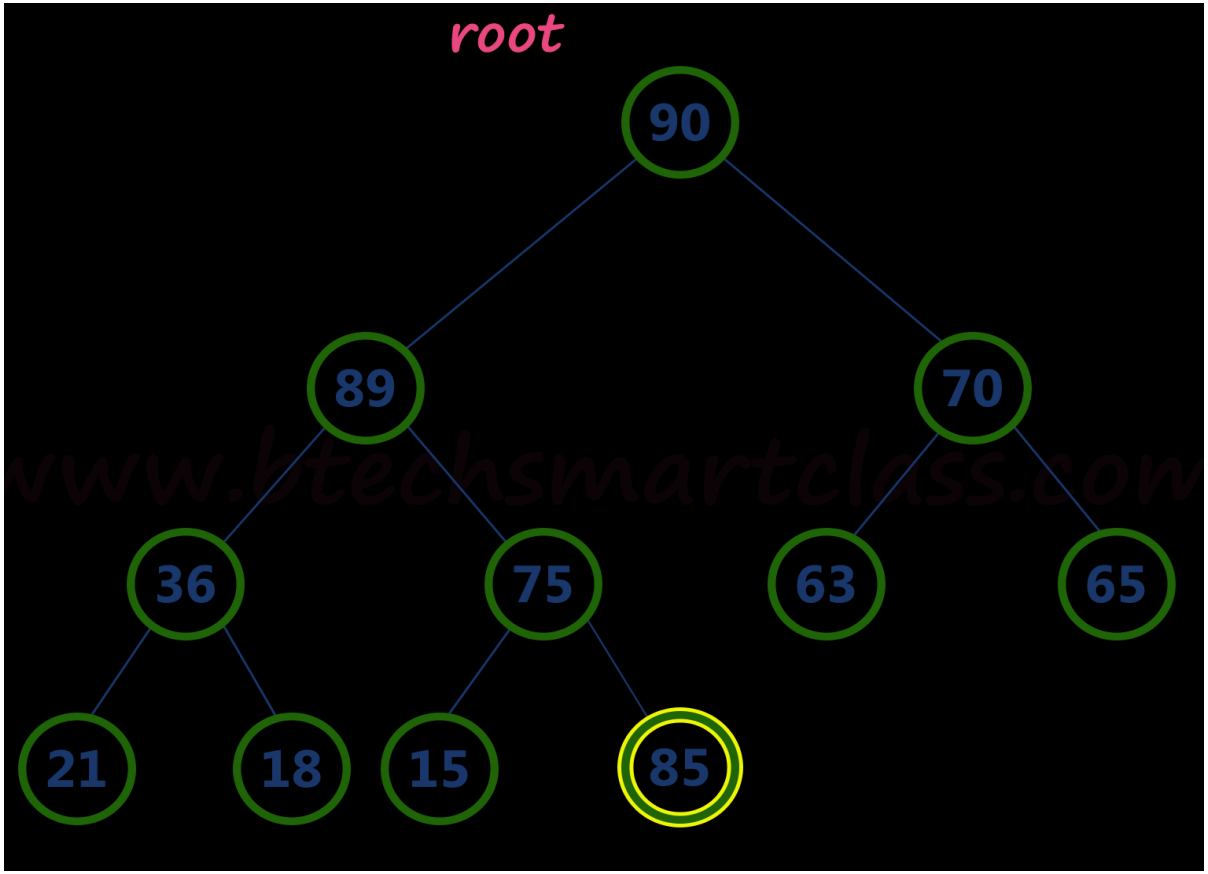
Max Heap

But now, this child node violates our heap-order property, because it is comparatively larger than its parent's value!

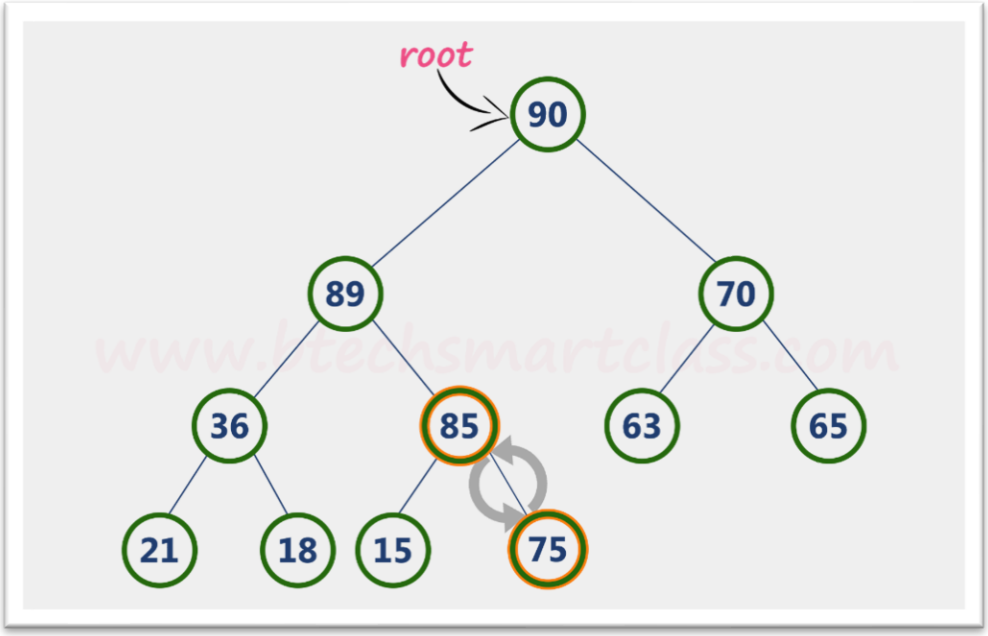


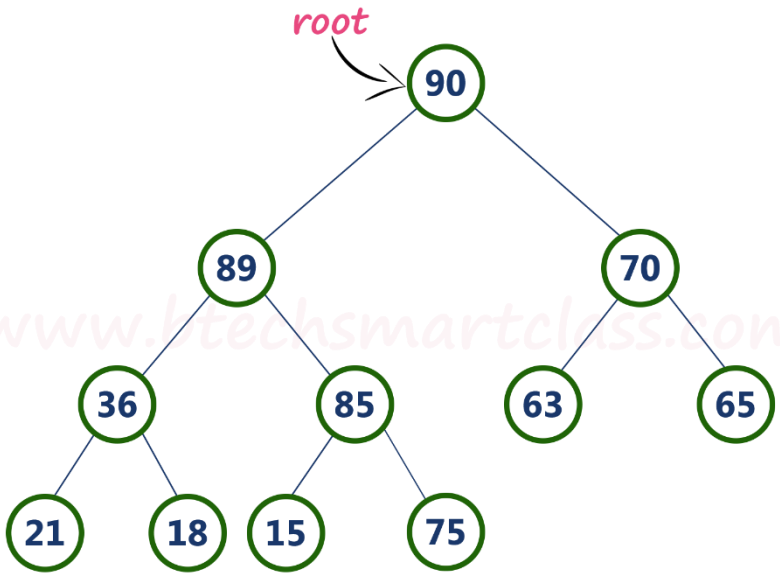
We can swap the two nodes, and now we're no longer violating our max heap condition! ✓

Insert a new node with value 85 for following tree.....



Is this a heap tree now ????????



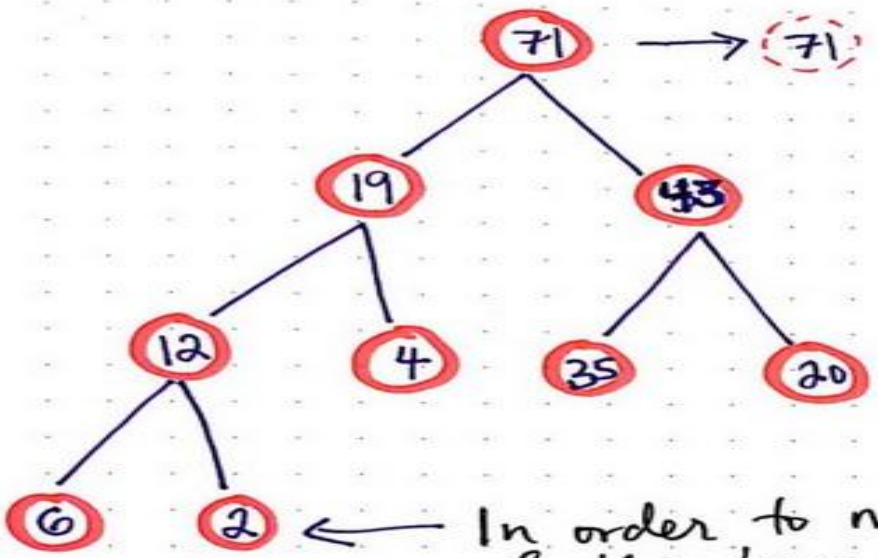


This is a heap tree now

Deletion operation (Shrinking the heap)

- When deleting or removing an element, most heaps are usually concerned with removing the root node.
- since the root will always be either the largest value element or the smallest value element, depending upon whether the heap is a max heap or **min** heap.

Shrinking the heap

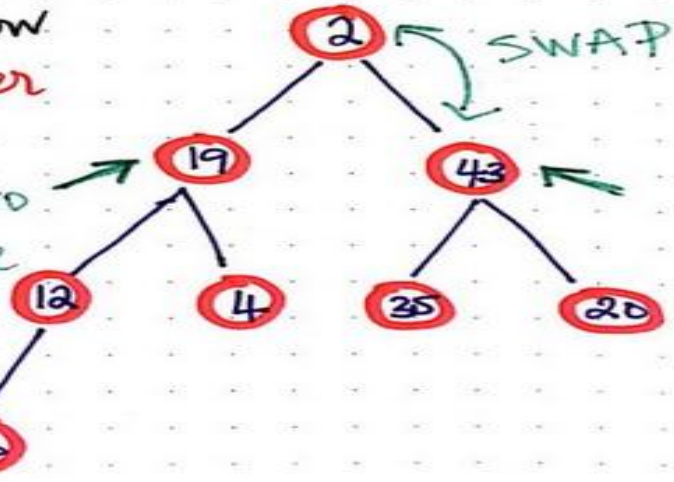


* Most heaps remove the root node, since they're usually concerned with the min or max value.

In order to maintain the shape/structure of this heap, the only we can remove and replace the root node with is the left-most, lowest-level leaf node.

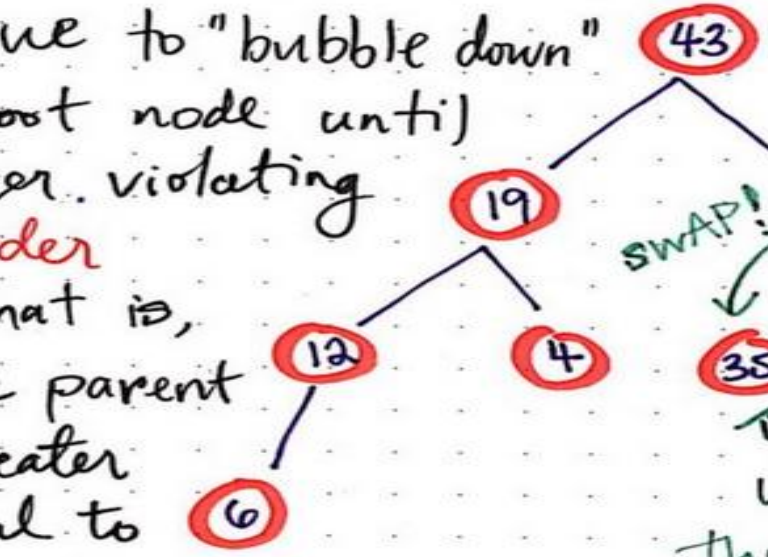
Our heap's shape is now correct, but our heap-order property is wrong!

We can compare the two children nodes of the root node, and swap the larger child node with the root node!

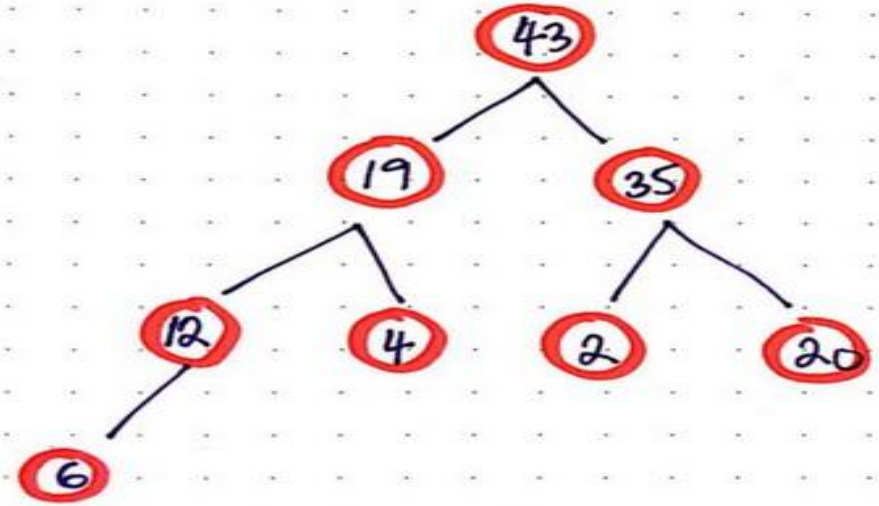


Shrinking the heap-Continue..

* We'll continue to "bubble down" the former root node until we're no longer violating the heap-order property — that is, until all the parent nodes are greater than or equal to their child nodes.



We'll swap the larger child element with its parent.

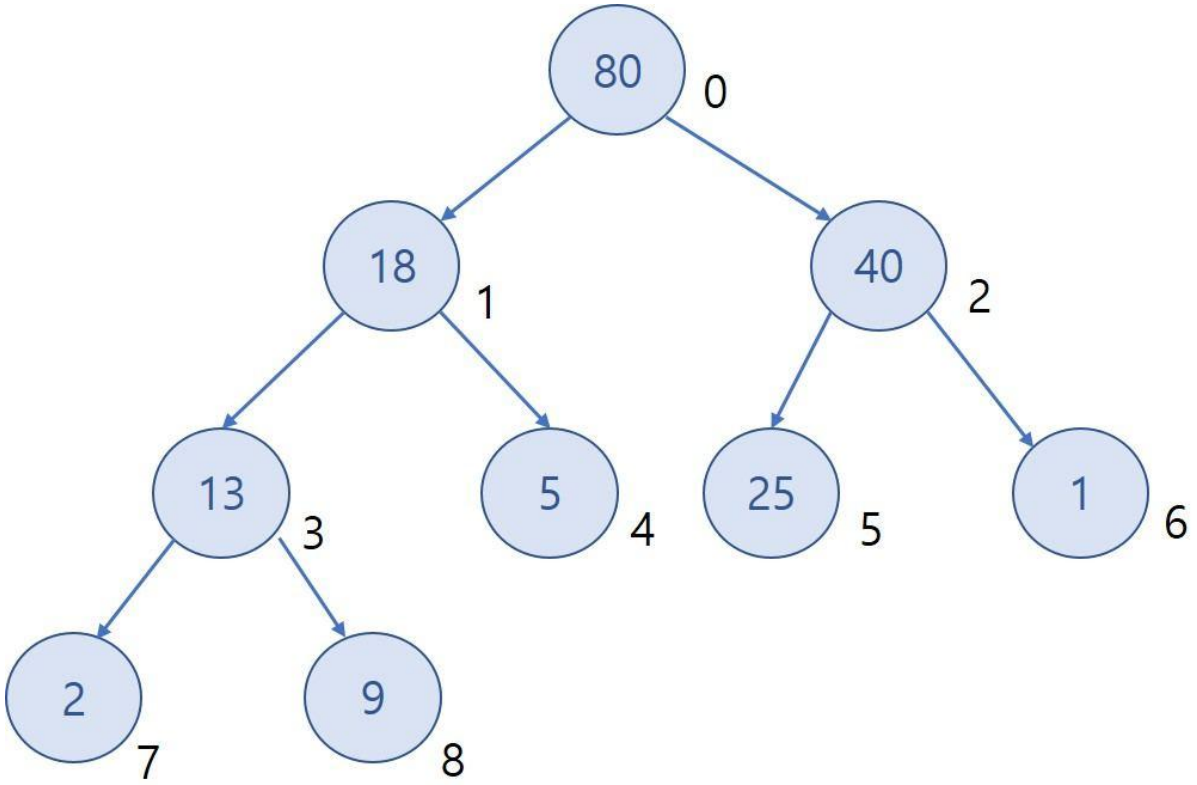


⟸ Our heap is now structured correctly, and is ordered correctly, too!

✓

Implementing the Heap using Arrays

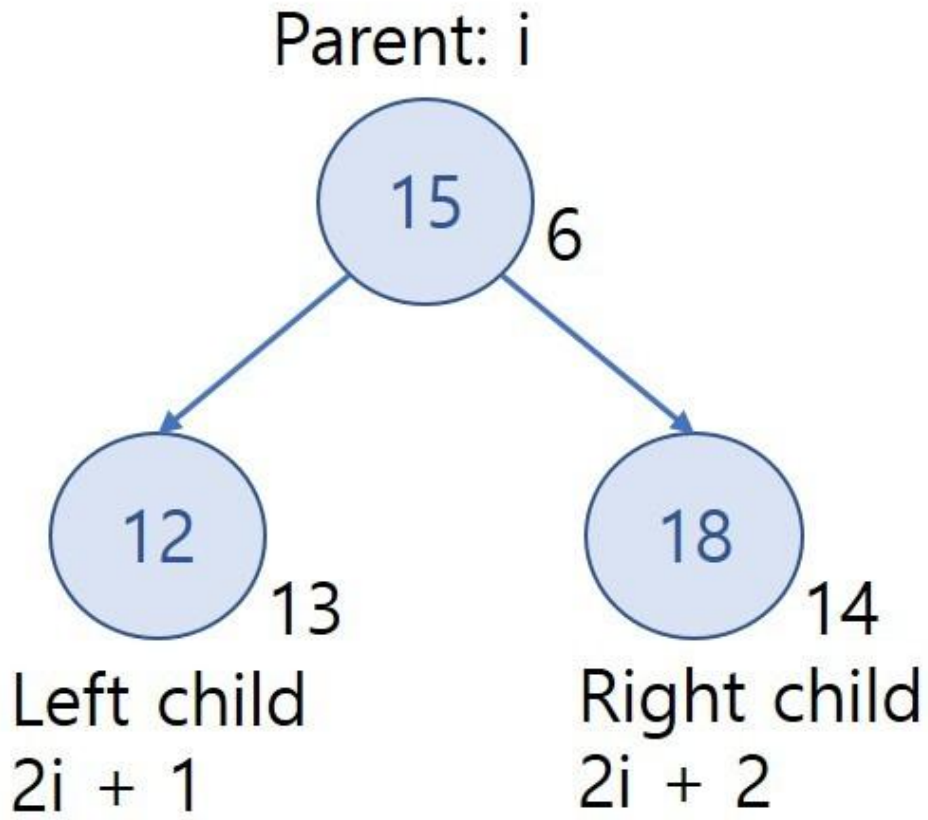
maximum heap.



Array representation.

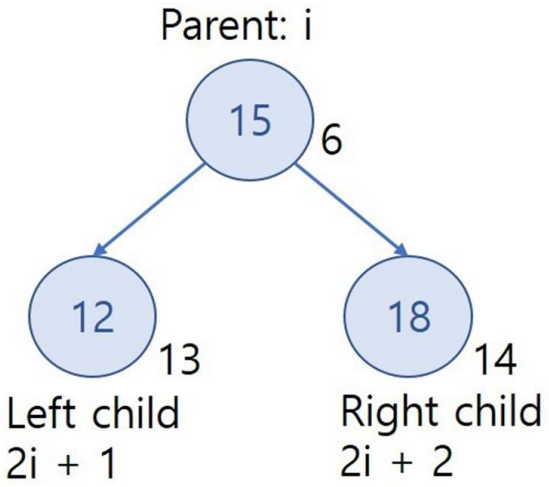
0	1	2	3	4	5	6	7	8
80	18	40	13	5	25	1	2	9

When we are building the heap using arrays as the underlying data structure, we need to be aware of the following formula for calculating the index of items in the tree.



Consider this array

0	1	2	3	4	5	6	7	8
80	18	40	13	5	25	1	2	9



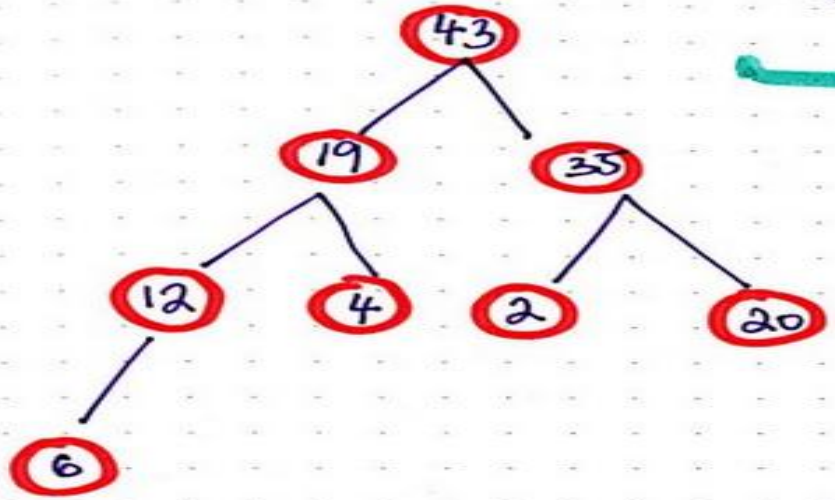
[80, 18, 40]
The index of 80 is 0. Therefore, i in this case equals zero. No problem understanding this right?

If **i** is zero, the index of 18 is
(2 * 0) + 1
0 + 1
1

Working with 40, its index will be

(2 * 0) + 2
0 + 2
2

Representing a heap as an array

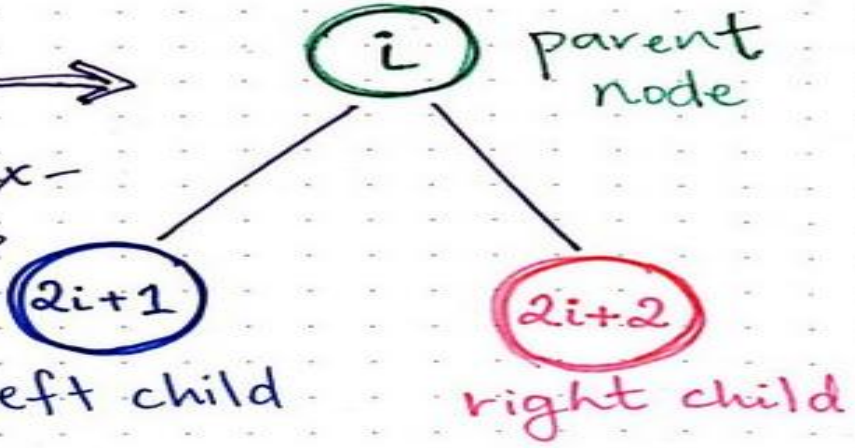


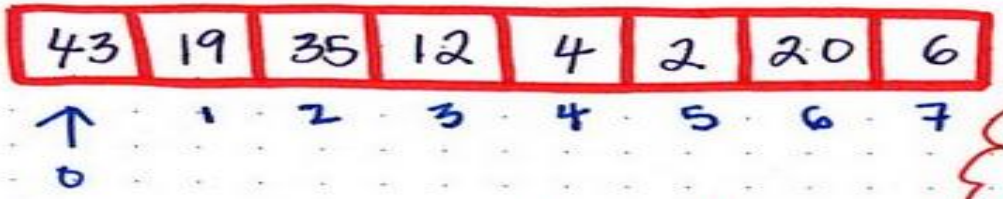
* If we know the index of the root node, we can manipulate that index to determine where its child nodes live in the array.

43	19	35	12	4	2	20	6

root node is always at index 0

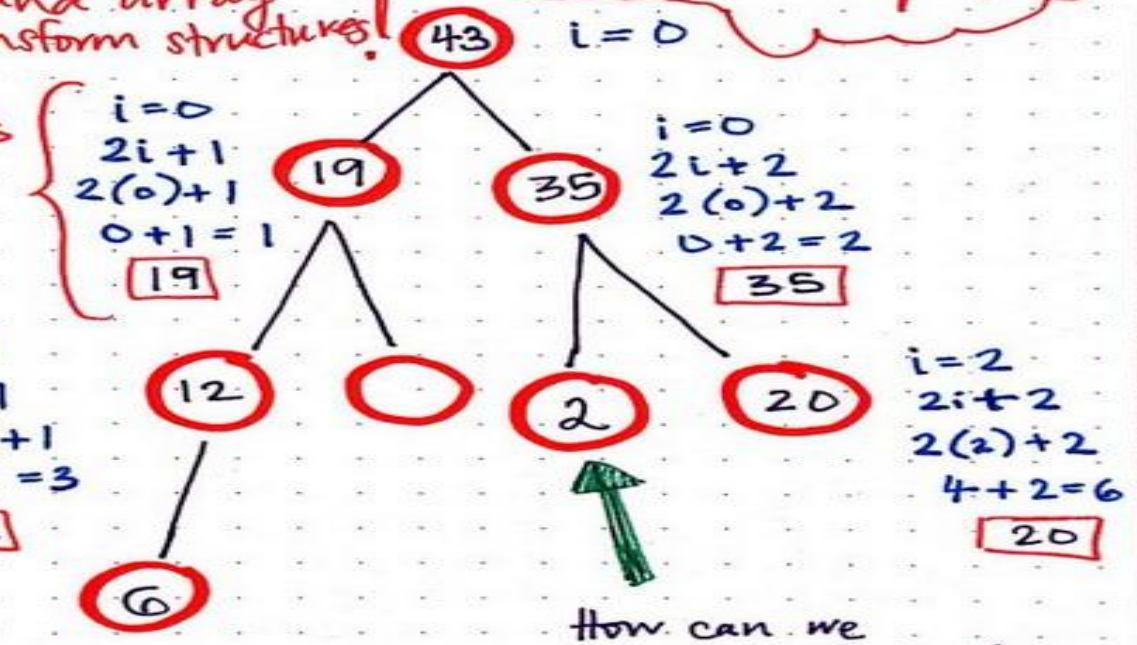
We can use index-based algorithms to determine which elements are child nodes of other elements in the array.





How can we represent an array as a help?

We can use parent-child relationships and array indexes to transform structures!



The parent node's index i is 0.

i=0
2i+1
2(0)+1
0+1=1
19

i=0
2i+2
2(0)+2
0+2=2
35

The parent node's index i is 1.

i=1
2i+1
2(1)+1
2+1=3
12

i=2
2i+2
2(2)+2
4+2=6
20

The parent node's index i is 3.

i=3
2i+1
2(3)+1
6+1=7
6

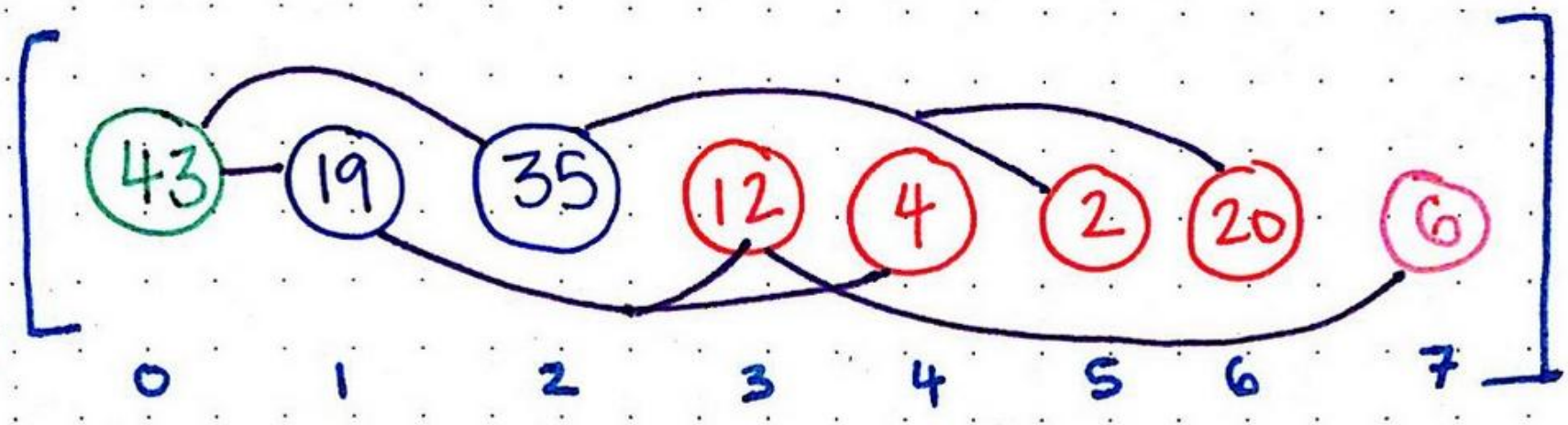
How can we determine the index of this ~~item's~~ node's parent?

The parent node of 2 is located at index 2: 35.

index of parent node $= \left\lfloor \frac{(n-1)}{2} \right\rfloor$ ← floor function shorthand

n = current node's index

$$\left\lfloor \frac{5-1}{2} \right\rfloor = \frac{4}{2} = 2$$



if i = index of a parent node —

$2i + 1$ = index of right child node

$2i + 2$ = index of left child node

if n = index of current node —

$\lfloor (n-1)/2 \rfloor$ = index of current node's parent.

Heap Sort

Procedures on Heap

- Heapify
- Build Heap
- Heap Sort

Heapify

- Heapify picks the largest child key and compare it to the parent key. If parent key is larger than heapify quits, otherwise it swaps the parent key with the largest child key. So that the parent is now becomes larger than its children.

```
Heapify(A, i)
{       l ← left(i)
      r ← right(i)
      if l <= heapsize[A] and A[l] > A[i]
         then largest ←l
         else largest ← i
      if r <= heapsize[A] and A[r] > A[largest]
         then largest ← r
      if largest != i
         then swap A[i] ←→ A[largest]
            Heapify(A, largest)   }
```

BUILD HEAP

- We can use the procedure 'Heapify' in a bottom-up fashion to convert an array A[1 . . *n*] into a heap. Since the elements in the subarray A[*n*/2 +1 . . *n*] are all leaves, the procedure BUILD_HEAP goes through the remaining nodes of the tree and runs 'Heapify' on each one. The bottom-up order of processing node guarantees that the subtree rooted at children are heap before 'Heapify' is run at their parent.

```
Buildheap(A)
{
    heapsize[A] ←length[A]
    for i ←|length[A]/2  //down to 1
       do Heapify(A, i)
}
```

Heap Sort Algorithm

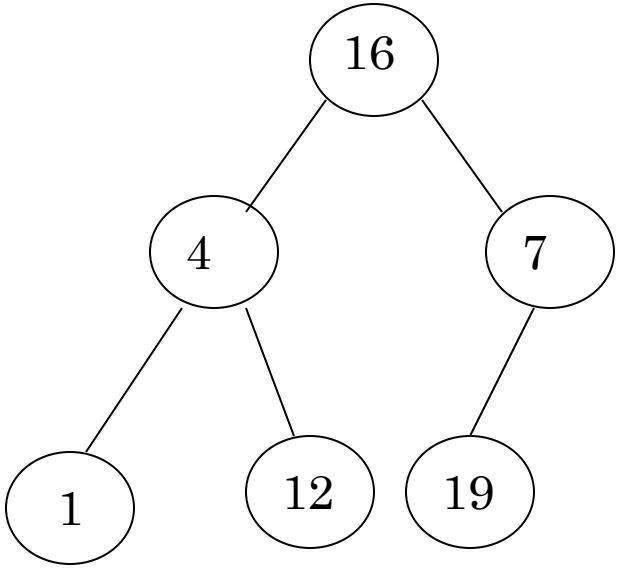
- The heap sort algorithm starts by using procedure BUILD-HEAP to build a heap on the input array A[1 . . *n*]. Since the maximum element of the array stored at the root *A*[1], it can be put into its correct final position by exchanging it with *A*[*n*] (the last element in *A*). If we now discard node n from the heap than the remaining elements can be made into heap. Note that the new element at the root may violate the heap property. All that is needed to restore the heap property.

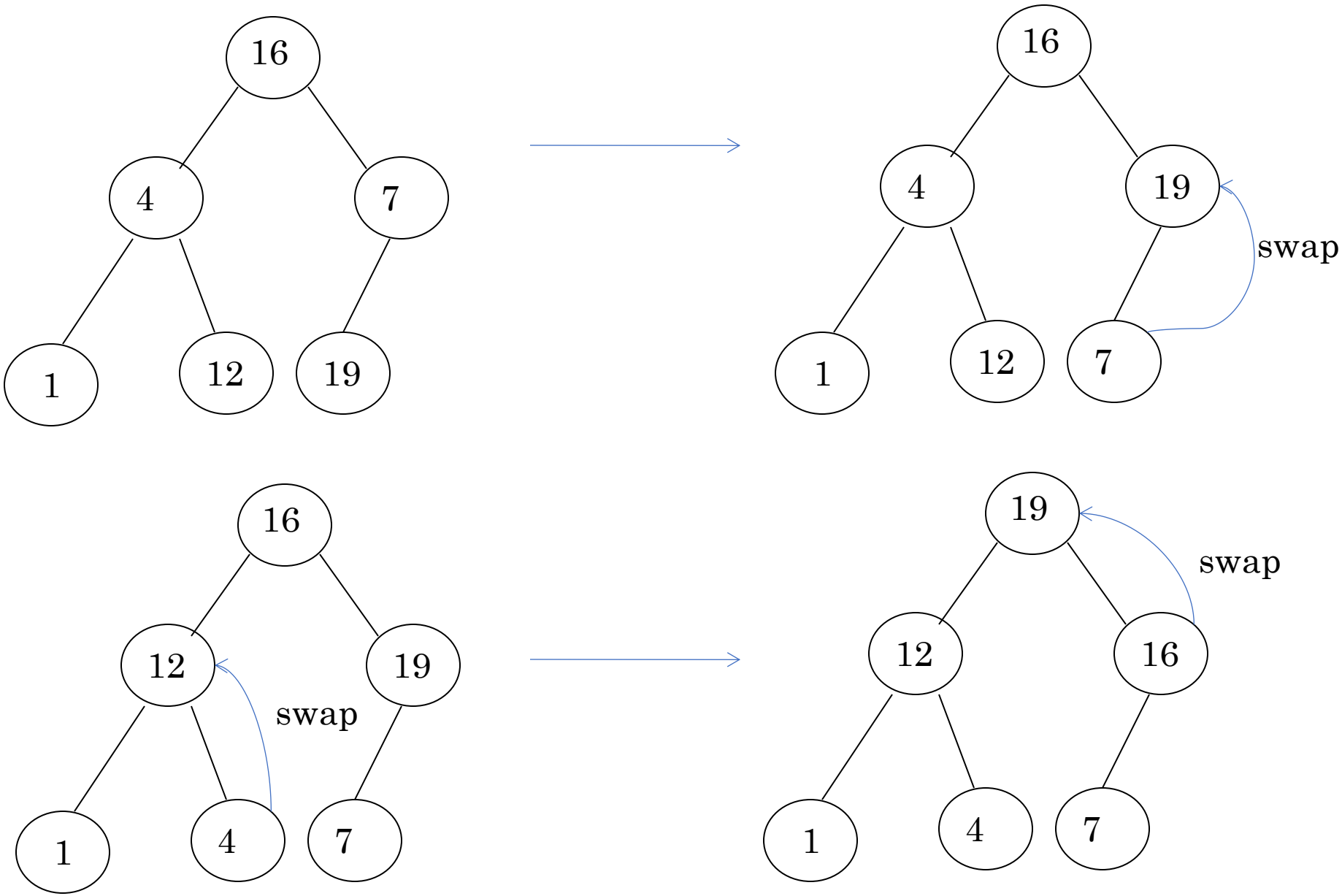
```
Heapsort(A)
{
      Buildheap(A)
      for i ← length[A] //down to 2
         do swap A[1] ←→ A[i]
         heapsize[A] ← heapsize[A] - 1
         Heapify(A, 1)
}
```

Example: Convert the following array to a heap

16	4	7	1	12	19

Picture **the array as a complete binary tree:**

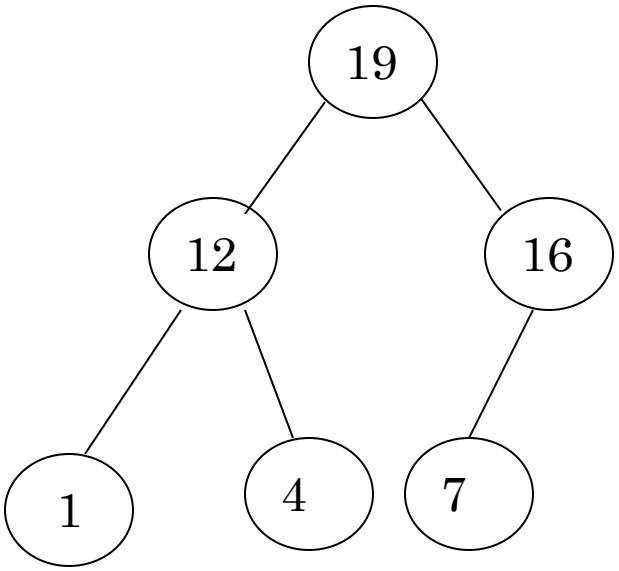



16
4
7
1
12
19
16
4
19
swap
1
12
7
16
12
19
swap
1
4
7
19
swap
12
16
1
4
7

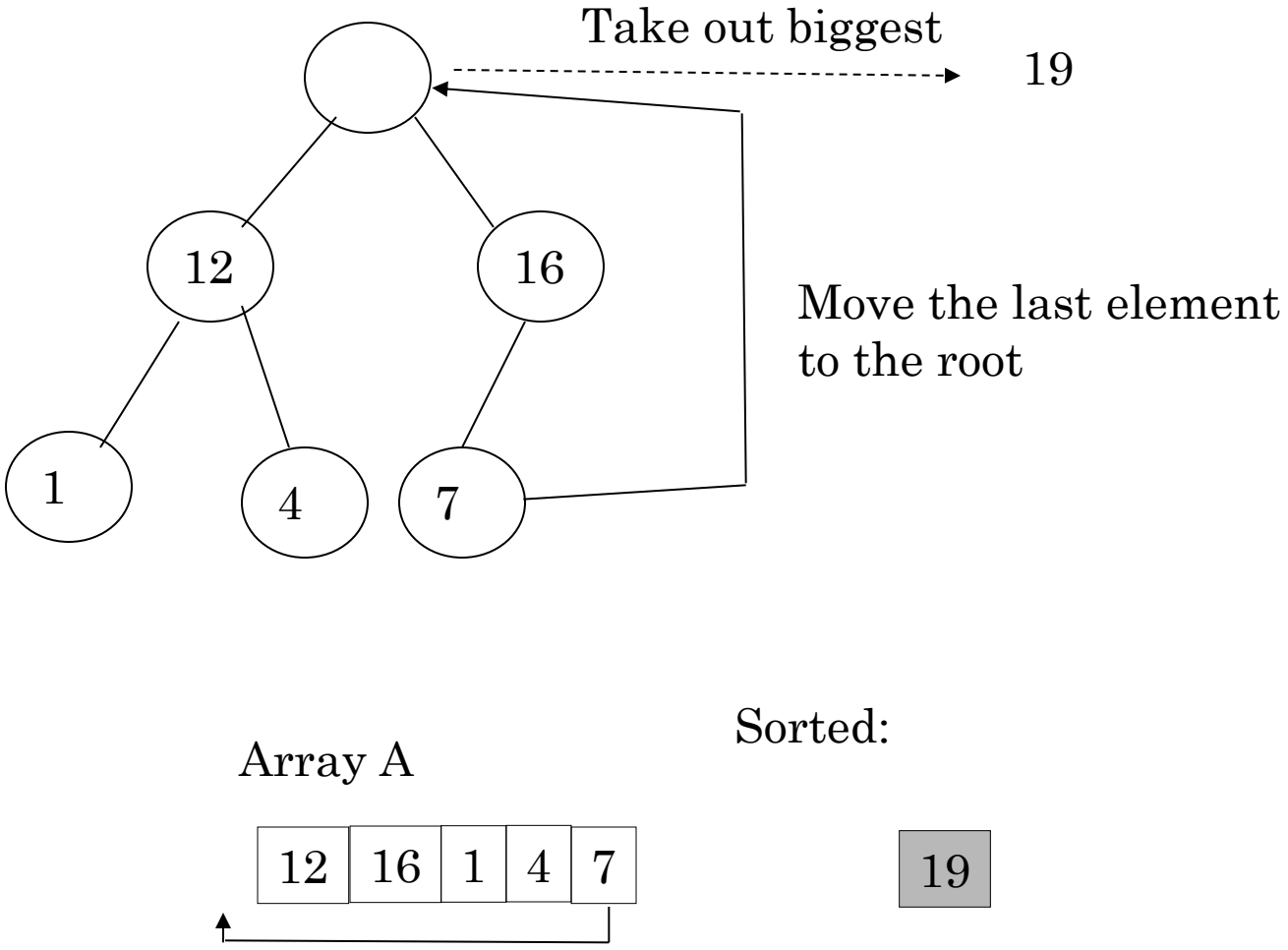
Heap Sort

- The heapsort algorithm consists of two phases:
  - build a heap from an arbitrary array
  - use the heap to sort the data

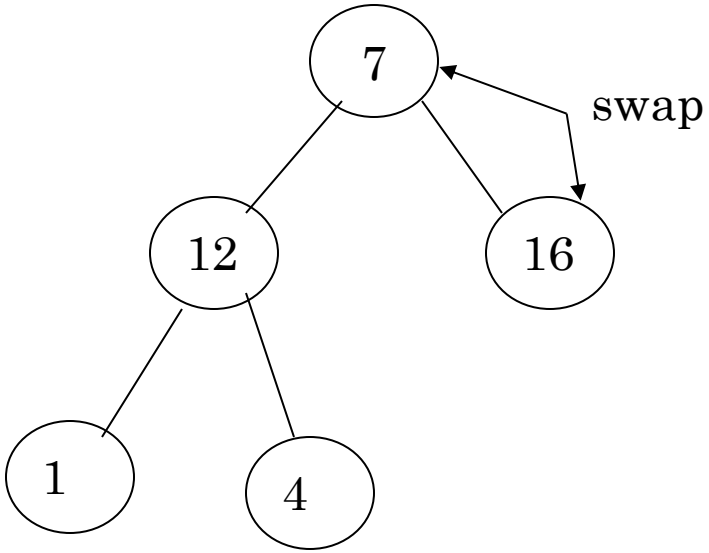
- To sort the elements in the decreasing order, use a min heap
- To sort the elements in the increasing order, use a max heap



Example of Heap Sort



HEAPIFY()

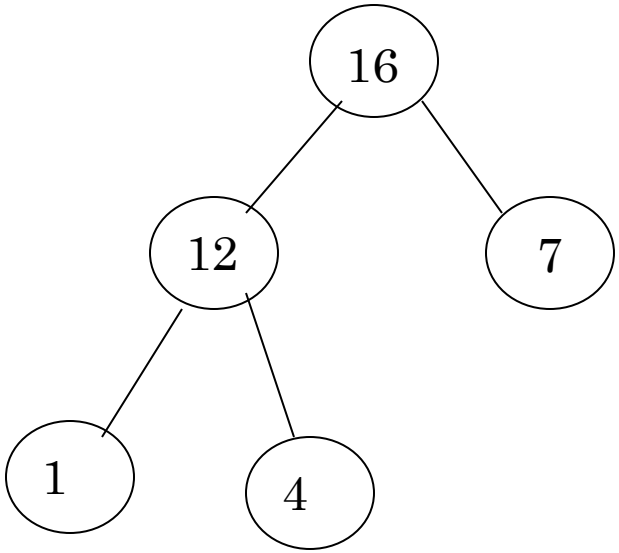


Array A

7	12	16	1	4

Sorted:

19

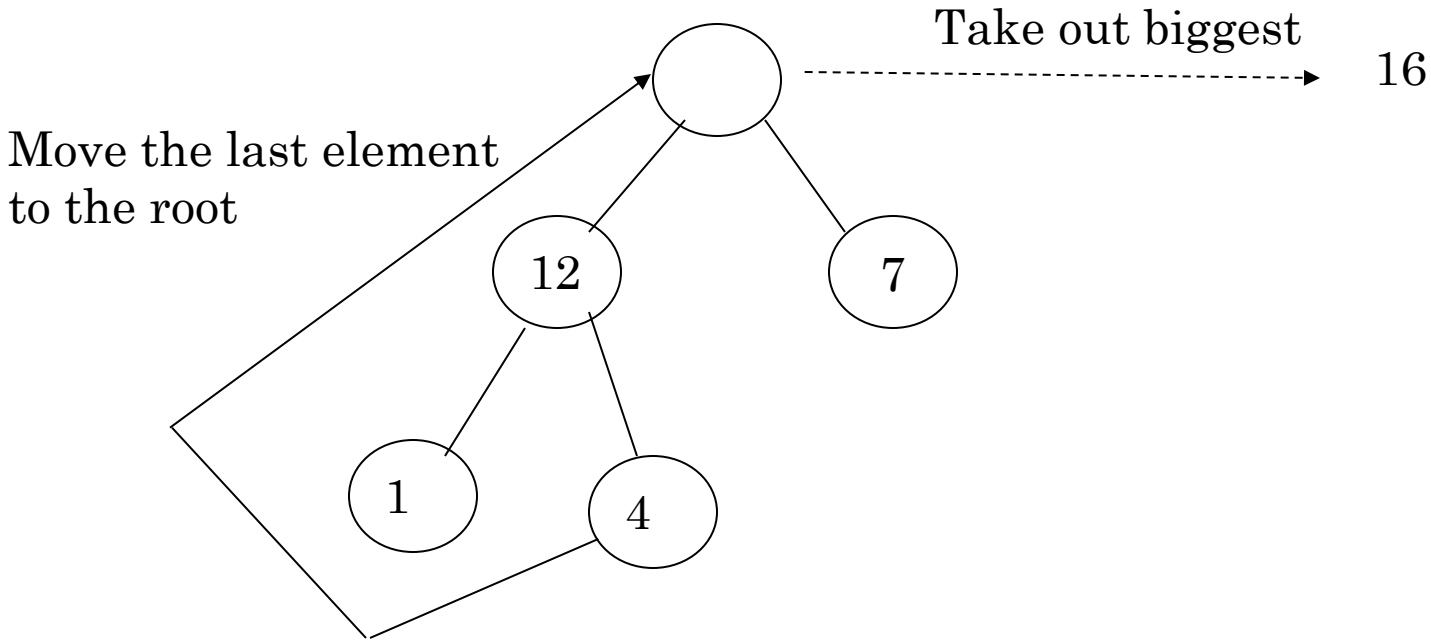


Array A

16	12	7	1	4

Sorted:

19

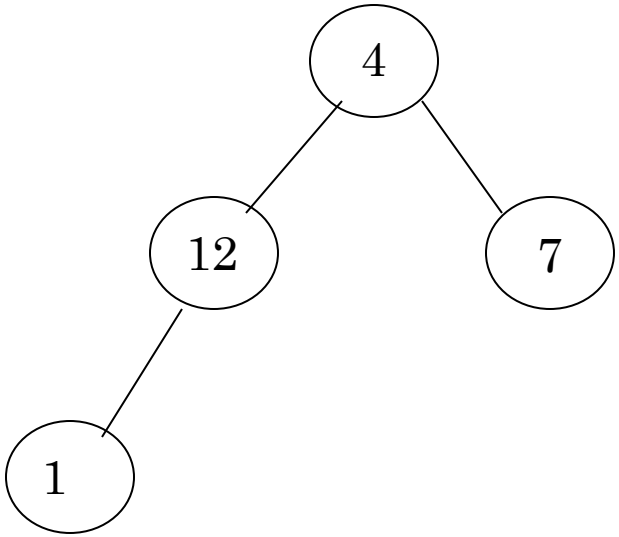


Sorted:

Array A

12	7	1	4

16	19



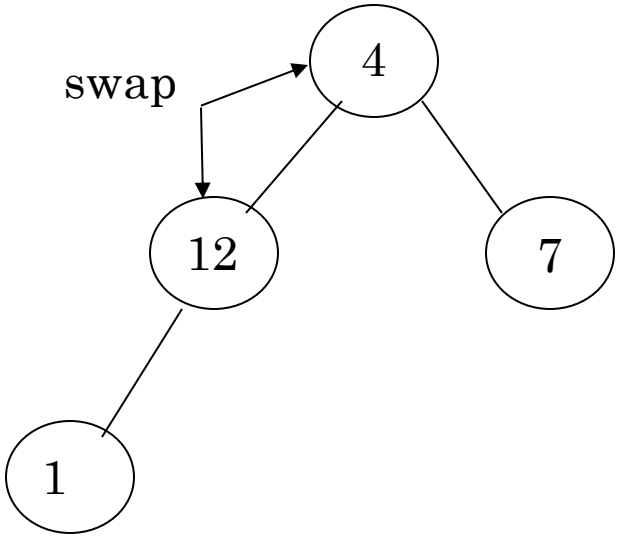
Array A

4	12	7	1

Sorted:

16	19

HEAPIFY()

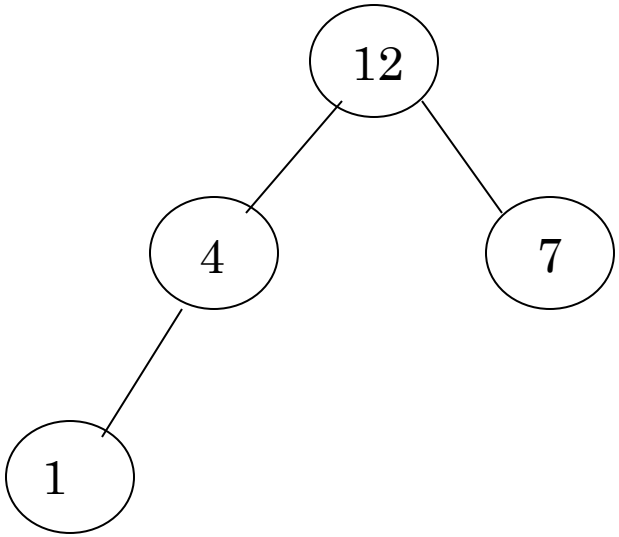


Array A

4	12	7	1

Sorted:

16	19

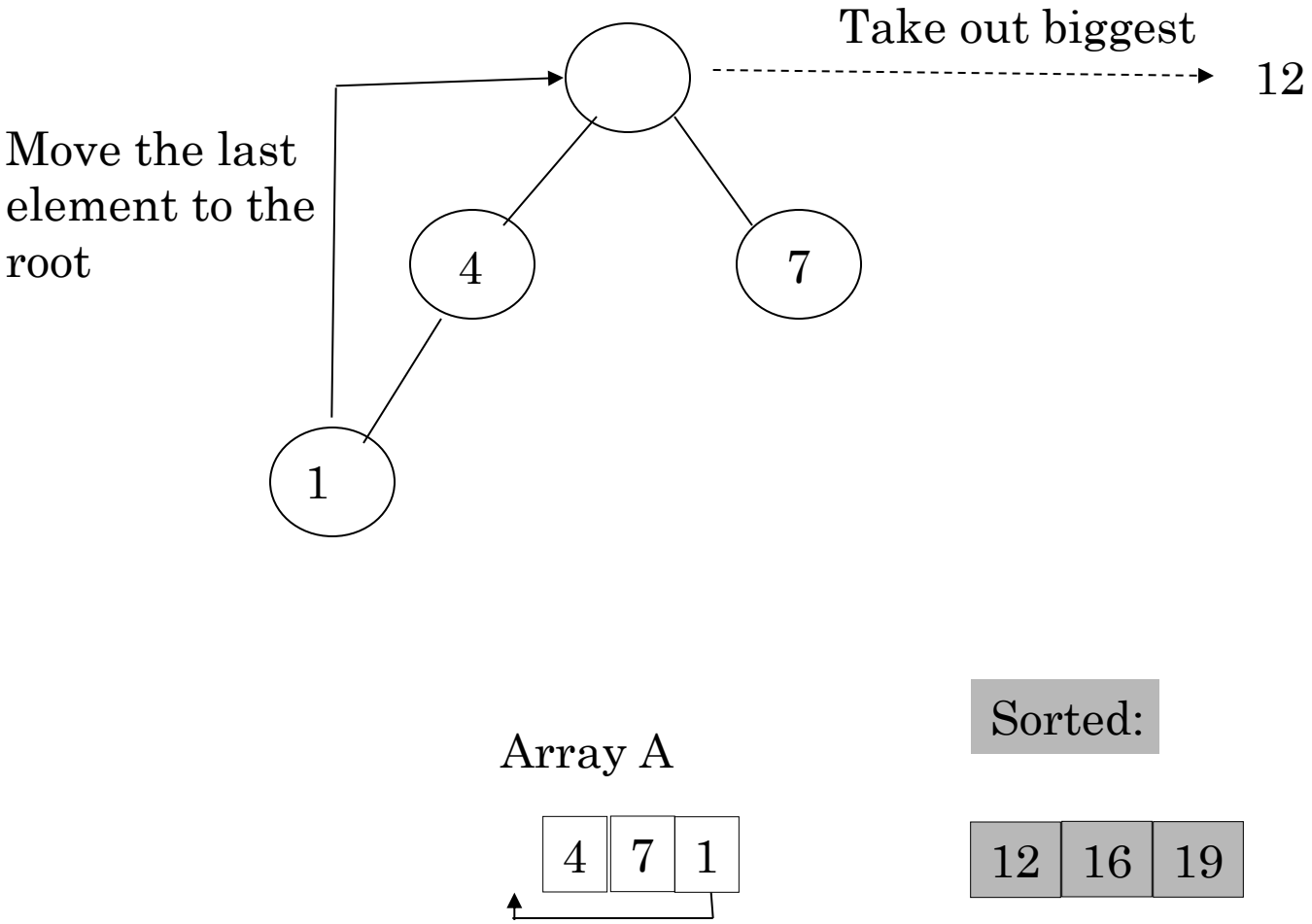


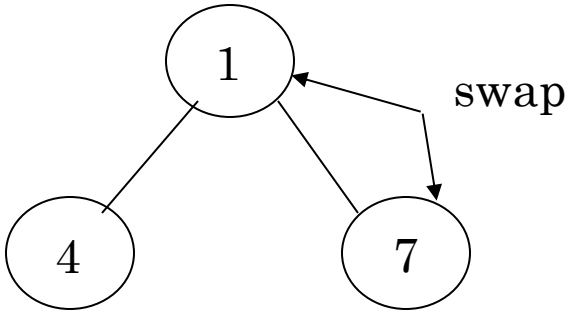
Array A

12	4	7	1

Sorted:

16	19


Take out biggest
12
Move the last element to the root
4
7
1
Array A
4
7
1
Sorted:
12
16
19

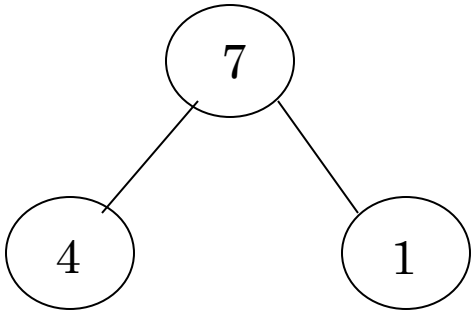


Array A

1	4	7

Sorted:

12	16	19

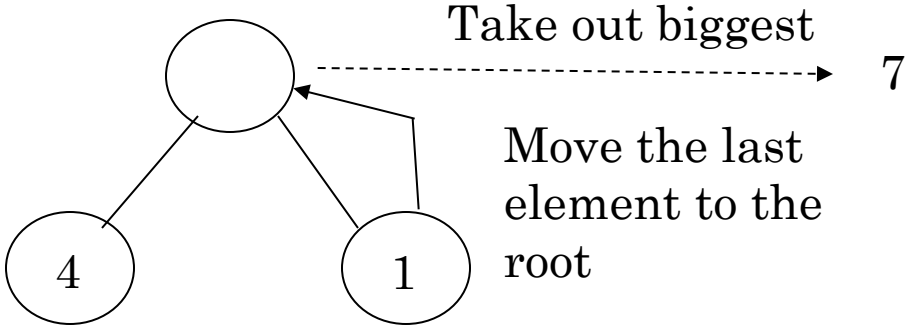


Array A

7	4	1

Sorted:

12	16	19



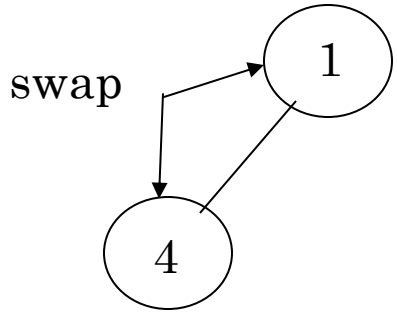
Array A

1	4

Sorted:

7	12	16	19

HEAPIFY()



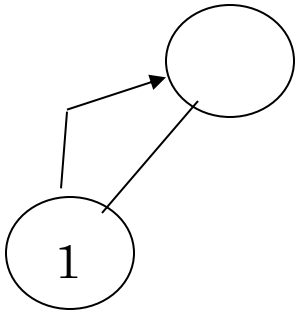
Array A

4	1

Sorted:

7	12	16	19

Move the last element to the root



Take out biggest

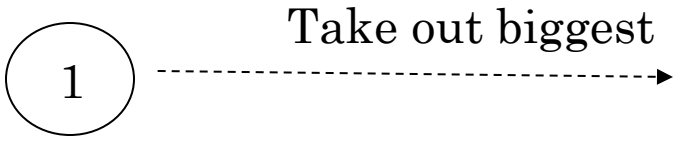
4

Array A

1

Sorted:

4	7	12	16	19

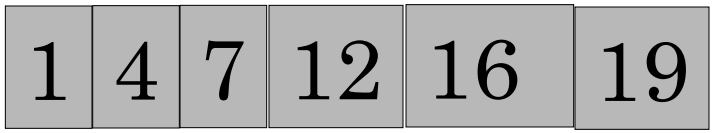


Array A

Sorted:

1	4	7	12	16	19

Sorted:



Complexity Analysis

- Worst Case Time Complexity: **O(n*log n)**
- Best Case Time Complexity: **O(n*log n)**
- Average Time Complexity: **O(n*log n)**
- Space Complexity : **O(1)**

Possible Application

- When we want to know the task that carry the highest priority given a large number of things to do
- Interval scheduling, when we have a lists of certain task with start and finish times and we want to do as many tasks as possible
- Sorting a list of elements that needs and efficient sorting algorithm

Conclusion

- The primary advantage of the heap sort is its efficiency. The execution time efficiency of the heap sort is O(n log n). The memory efficiency of the heap sort, unlike the other n log n sorts, is constant, O(1), because the heap sort algorithm is not recursive.
- The heap sort algorithm has two major steps. The first major step involves transforming the complete tree into a heap. The second major step is to perform the actual sort by extracting the largest element from the root and transforming the remaining tree into a heap.

END